ST.THOMAS INSTITUTE FOR SCIENCE AND TECHNOLOGY

FIRE: Food Image to Recipe Generation

GUIDED BY :
MRS. BINDHYA PS
ASST. PROFESSOR, CSE

SEMINAR PRESENTATION

DONE BY :
NAVAFA NOUSHAD
STI21CS047

CONTENT

1 INTRODUCTION

2 CHALLENGES IN COOKING TODAY

3 LITERATURE SURVEY

3.1 DRAWBACKS

4 PROPOSED SOLUTIONS

4.1 FIRE Model Overview

5 MODEL ARCHITECTURE

5.1 Technical Components of FIRE

6 WORKING PROCESS & ALGORITHMS

1. Title Generation
2. Ingredient Extraction
3. Cooking Instruction Generation

7 TRAINING PROCESS

8 PERFORMANCE MATRICES

9 KEY FEATURES & BENEFITS

10 CONCLUSION

11 REFERENCES

Introduction to Food Computing

- Definition: Food computing is a multidisciplinary field that leverages AI to address food-related tasks such as recipe generation from images.
- Significance: Food is deeply tied to culture, health, and identity. With millions of food images shared daily on social media, the need for automatic systems to generate recipes from these images has grown
- Challenge: Current models face limitations due to dataset size and lack of integration between vision and language models

Challenges in Cooking Today

- Difficulty in meal planning and finding new recipes.
- Managing dietary restrictions and preferences.
- Reducing food waste by using available ingredients.
- Existing food journaling tools require significant user input, leading to low adherence over time.
- Traditional methods often lack precision in portion size estimation and nutrient calculation.

Literature Survey

Martinel et al. (2018) - Introduced a slice convolution block in residual networks (ResNet) to capture features from food images

Salvador et al. (2019) - Proposed InverseCooking, an encoder-decoder framework to generate food titles and ingredient lists from food images

Wang et al. (2020) - Treated recipe generation as an image captioning task and used tree-structured representations to improve recipe coherence

Salvador et al. (2019) - InverseCooking:

- Strengths: It uses a transformer-based model for ingredient extraction and cooking instruction generation.

Chef Transformer (2020) - Focused on generating recipes from ingredient lists, not images

CLIP (2021) - Utilized a contrastive learning approach to align image and text representations.

Xie et al. (2010) - Used a similarity and filtering algorithm to retrieve recipes based on the cooking flow

Draw Backs

- Focused primarily on global features, neglecting local features critical for ingredient detection.
- Struggles with images where ingredients overlap or are occluded
- Overemphasized global image features, leading to errors in identifying overlapping or hidden ingredients.
- Still reliant on retrieval-based approaches, which limit creativity and generalization.
- Doesn't handle ingredient extraction effectively for unseen or complex dishes.
- Limited to pre-defined ingredients provided during testing, rather than extracting ingredients from images.
- Fails to handle cases where the same ingredients lead to different recipes, thus requiring human input for context.
- Doesn't address the generation of complex recipes where ingredient interactions are intricate.

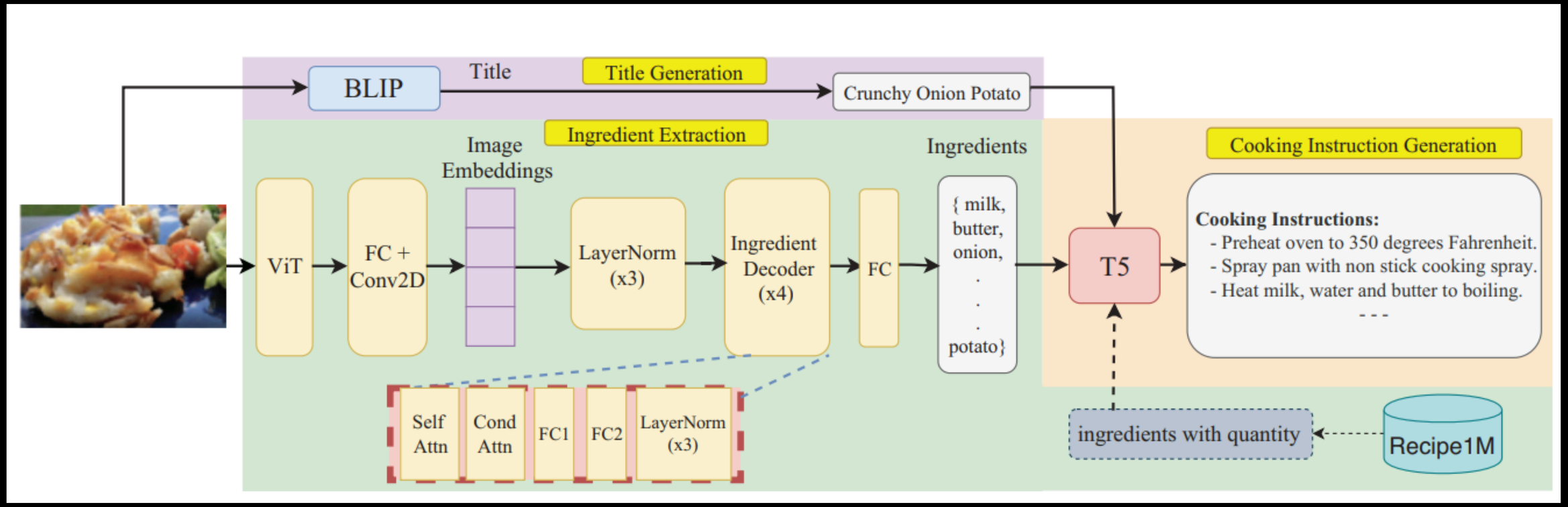
FIRE Model Overview

- FIRE Objective: Generate food titles, ingredients, and cooking instructions from a given food image.

- Key Contributions:
  a. Vision Transformers (ViT) for ingredient extraction.
  b. BLIP Model for title generation.
  c. T5 Model for cooking instructions

Model Architecture of FIRE

- FIRE (Food Image to REcipe generation) leverages a multimodal approach combining computer vision for image understanding and natural language models for text generation.
- The core architecture of FIRE consists of three key components:
  - Title Generation using an image captioning model.
  - Ingredient Extraction using a Vision Transformer (ViT).
  - Cooking Instruction Generation using the T5 language model.


BLIP
Title
Title Generation
Crunchy Onion Potato
Ingredient Extraction
Image Embeddings
Ingredients
Cooking Instruction Generation
ViT
FC + Conv2D
LayerNorm (x3)
Ingredient Decoder (x4)
FC
{ milk, butter, onion, . . . potato}
T5
Cooking Instructions:
- Preheat oven to 350 degrees Fahrenheit.
- Spray pan with non stick cooking spray.
- Heat milk, water and butter to boiling.
- - -
Self Attn
Cond Attn
FC1
FC2
LayerNorm (x3)
ingredients with quantity
Recipe1M

Technical Components of FIRE

- Title Generation:
  - BLIP Model: Originally developed for general image captioning, it was fine-tuned to produce concise recipe titles from food images.
  - Improvement: Fine-tuning BLIP with food-specific datasets increased title accuracy
- Ingredient Extraction:
  - Vision Transformer (ViT): Extracts features from food images, followed by a custom decoder for ingredient extraction.
  - Key Innovation: FIRE predicts ingredients using a set-based approach, meaning the order of ingredients does not affect the result

Technical Components of FIRE

- Cooking Instruction Generation:
  - T5 Language Model: Generates natural-language cooking instructions by using both the recipe title and the extracted ingredients as inputs.
  - Challenge Solved: T5 handles the absence of ingredient quantities at inference time

Title: Crunchy Onion Potato

Ingredients: Milk, Butter, Onion, ..., Potato

Cooking Instructions:

- Preheat oven to 350 degrees Fahrenheit.
- Spray pan with non stick cooking spray.
- Heat milk, water and butter to boiling.
- Stir in contents of both pouches of potatoes.
- Let stand one minute. Stir in corn.
- Spoon half the potato mixture in pan.
- Sprinkle half each of cheese and onions.
- Top with remaining potatoes.
- Sprinkle with remaining cheese and onions.
- Bake 10 to 15 minutes until cheese is melted.

Step-by-Step Process of FIRE

Step 1: Title Generation

- Model Used: BLIP (Bootstrapping Language-Image Pre-training).
- Input: A food image.
- Output: A short, descriptive title for the dish.
- Process:
  - The input image is passed through a pre-trained image-captioning model, BLIP, which is fine-tuned on a food-specific dataset.
  - The BLIP model generates captions describing the image.
  - Fine-tuning helps the model focus on food-related titles rather than general captions (e.g., removing background or irrelevant information from the title).

Algorithm

- Fine-tuned BLIP model learns to minimize the difference between predicted and ground-truth titles using cross-entropy loss during training.
- The model uses a transformer-based encoder-decoder architecture to generate captions, where the encoder extracts image features, and the decoder generates the title.

Step-by-Step Process of FIRE

Step 2: Ingredient Extraction

- Model Used: Vision Transformer (ViT) with an attention-based decoder.
- Input: The same food image.
- Output: A list of ingredients (in set form, without quantity at this stage).
- Process:
  - The input image is processed through a Vision Transformer (ViT), which extracts features from different regions of the image using self-attention mechanisms.
  - These features are passed through a decoder that predicts ingredients.
  - The decoder is conditioned on the extracted visual features and uses a multi-layer attention mechanism to output the relevant ingredients.

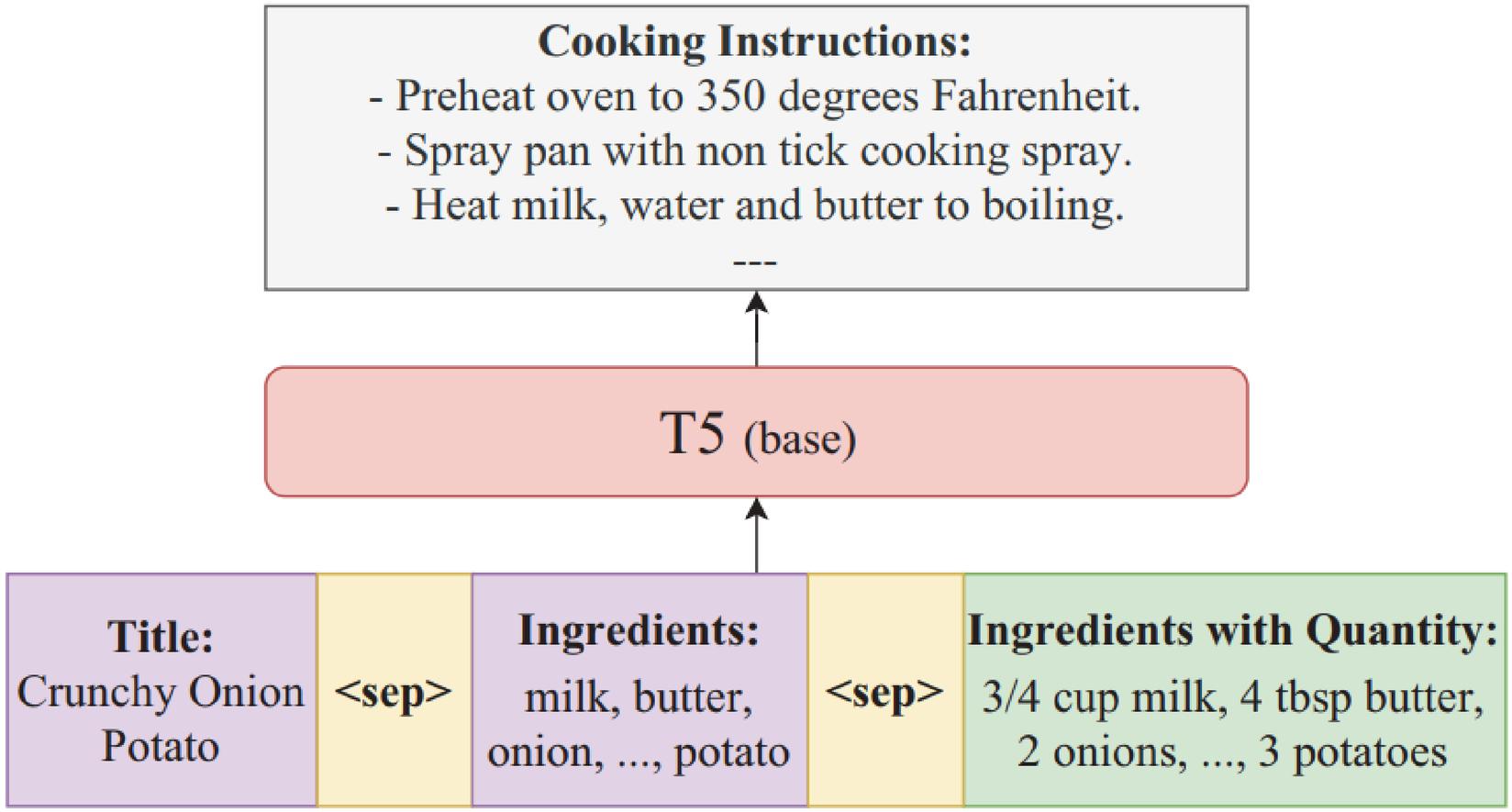
Algorithm

- ViT uses a series of self-attention layers to capture both global and local image features.
- The ingredient extraction decoder works by predicting each ingredient sequentially until it encounters an End-of-Sequence (EOS) token.
- The predicted ingredient list is represented as a binary vector, where each position corresponds to whether a particular ingredient is present (1) or absent (0) in the recipe.
- Loss Function: The model minimizes a combination of binary cross-entropy loss (for correct ingredient prediction) and cardinality loss (which helps the model predict the correct number of ingredients).

Step-by-Step Process of FIRE

Step 3: Cooking Instruction Generation

- Model Used: T5 (Text-to-Text Transfer Transformer).
- Input: The generated title and the list of extracted ingredients.
- Output: Step-by-step cooking instructions in natural language.
- Process:
  - The T5 model is fine-tuned to generate cooking instructions based on a formatted input combining the dish title and the list of ingredients.
  - The T5 model works by taking a sequence of text inputs and generating another sequence (the instructions) as output.
  - The training is done using the Recipe1M dataset, where the model learns the relationships between ingredients, titles, and the sequence of instructions.


Cooking Instructions:
- Preheat oven to 350 degrees Fahrenheit.
- Spray pan with non tick cooking spray.
- Heat milk, water and butter to boiling.

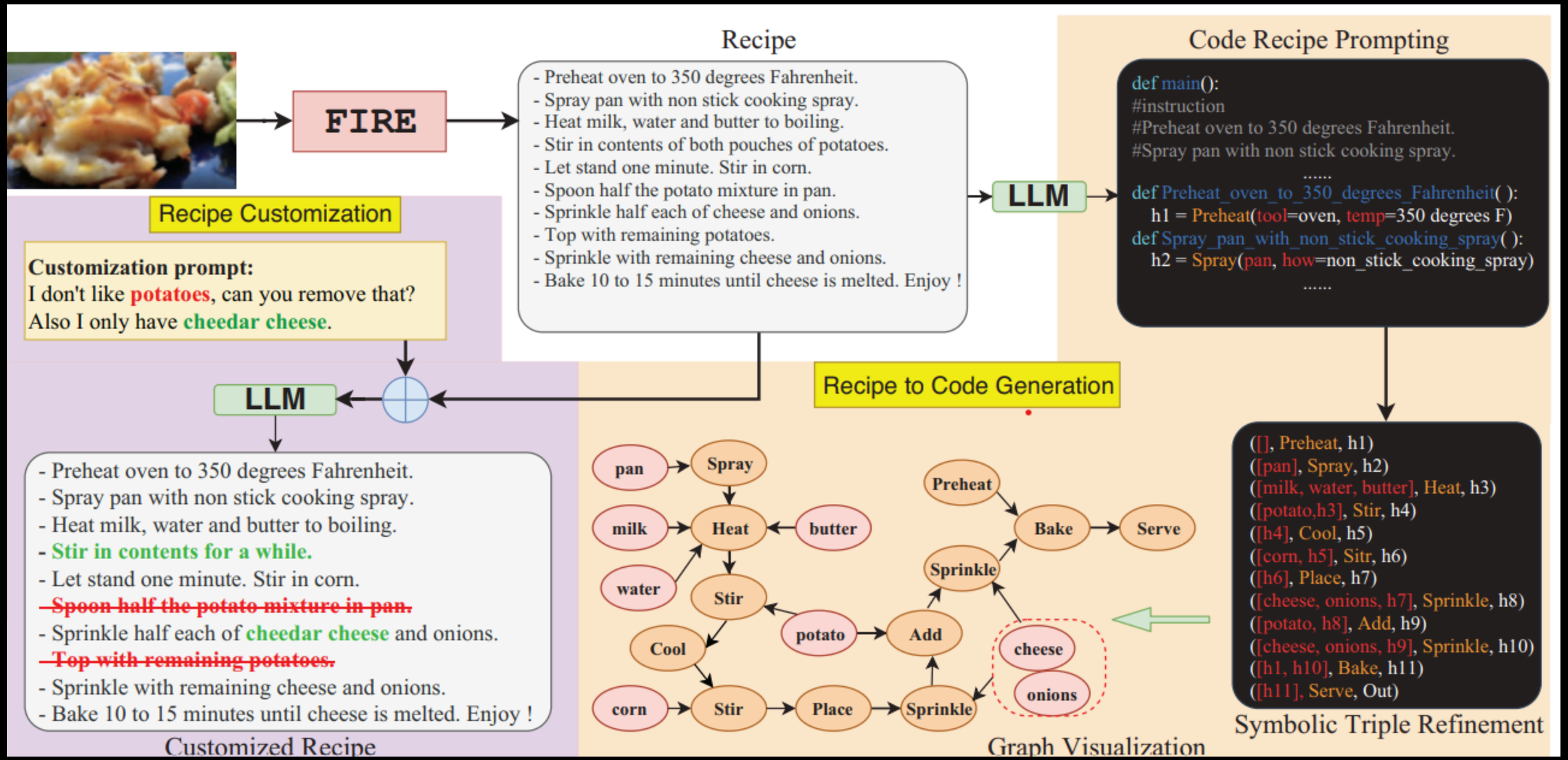
T5 (base)
Title:
Crunchy Onion Potato
<sep>
Ingredients:
milk, butter, onion, ..., potato
<sep>
Ingredients with Quantity:
3/4 cup milk, 4 tbsp butter, 2 onions, ..., 3 potatoes

Algorithm

- T5 uses an encoder-decoder transformer architecture:
  - The encoder processes the concatenated input (title + ingredients).
  - The decoder generates the instructions one step at a time, using self-attention to ensure that each part of the instruction is coherent.
- During inference, the model predicts cooking instructions based on the title and ingredients alone, as ingredient quantities are not available at test time.
- The model is trained using maximum likelihood estimation (MLE), where the predicted sequence of instructions is compared to the ground truth.

Training Process

1. Data Preprocessing:
   - Recipe1M Dataset: Contains recipes with titles, ingredients, and instructions, along with corresponding food images.
   - Preprocess the dataset by filtering for recipes with images, and normalize the text data for ingredients and instructions.
2. Model Training:
   - Fine-tuning BLIP: Train the BLIP model on food-related data, ensuring it generates concise and relevant recipe titles.
   - Training ViT: Train the Vision Transformer to recognize ingredient patterns in images. The binary cross-entropy loss helps optimize ingredient prediction.
   - Training T5: Fine-tune T5 to generate cooking instructions based on the extracted title and ingredients. Use SacreBLEU and ROUGE-L to evaluate the quality of generated instructions.


FIRE
Recipe
- Preheat oven to 350 degrees Fahrenheit.
- Spray pan with non stick cooking spray.
- Heat milk, water and butter to boiling.
- Stir in contents of both pouches of potatoes.
- Let stand one minute. Stir in corn.
- Spoon half the potato mixture in pan.
- Sprinkle half each of cheese and onions.
- Top with remaining potatoes.
- Sprinkle with remaining cheese and onions.
- Bake 10 to 15 minutes until cheese is melted. Enjoy !
LLM
Code Recipe Prompting
def main():
#instruction
#Preheat oven to 350 degrees Fahrenheit.
#Spray pan with non stick cooking spray.
.....
def Preheat_oven_to_350_degrees_Fahrenheit( ):
h1 = Preheat(tool=oven, temp=350 degrees F)
def Spray_pan_with_non_stick_cooking_spray( ):
h2 = Spray(pan, how=non_stick_cooking_spray)
.....
Recipe Customization
Customization prompt:
I don't like potatoes, can you remove that?
Also I only have cheedar cheese.
LLM
- Preheat oven to 350 degrees Fahrenheit.
- Spray pan with non stick cooking spray.
- Heat milk, water and butter to boiling.
- Stir in contents for a while.
- Let stand one minute. Stir in corn.
~~- Spoon half the potato mixture in pan.~~
- Sprinkle half each of cheedar cheese and onions.
~~- Top with remaining potatoes.~~
- Sprinkle with remaining cheese and onions.
- Bake 10 to 15 minutes until cheese is melted. Enjoy !
Customized Recipe
Recipe to Code Generation
pan
Spray
milk
Heat
butter
water
Stir
Cool
corn
Stir
Place
potato
Add
Sprinkle
cheese
onions
Preheat
Bake
Serve
Sprinkle
Graph Visualization
([], Preheat, h1)
([pan], Spray, h2)
([milk, water, butter], Heat, h3)
([potato,h3], Stir, h4)
([h4], Cool, h5)
([corn, h5], Sitr, h6)
([h6], Place, h7)
([cheese, onions, h7], Sprinkle, h8)
([potato, h8], Add, h9)
([cheese, onions, h9], Sprinkle, h10)
([h1, h10], Bake, h11)
([h11], Serve, Out)
Symbolic Triple Refinement

Performance Metrics

- Ingredient Extraction: Evaluated using F1-score and IoU (Intersection over Union), focusing on how accurately the model predicts the correct set of ingredients.
- Instruction Generation: Evaluated using SacreBLEU and ROUGE-L, which measure how similar the generated cooking instructions are to human-written ones.
- Ingredient extraction showed a relative improvement of 1.5% in IoU and 1.4% in F1 score compared to InverseCooking
- T5 generated higher-quality cooking instructions when fine-tuned compared to zero-shot prompting

Key Features and Benefits

- Prioritize recipes with low environmental impact through eco-friendly algorithms.
- Implement health filters with decision trees or rule-based systems.
- Efficiency and Time Savings
- Health and Nutrition
- Variety and Creativity
- Apply batch processing algorithms to generate meal plans quickly.
- Implement efficient indexing techniques for fast recipe search.
- Use reinforcement learning to refine recommendations based on user feedback.
- Implement constraint satisfaction algorithms for dietary restrictions.
- Use collaborative filtering and matrix factorization for preference modeling.
- Match recipes with current kitchen inventory using optimization algorithms.
- Track user progress and provide feedback on cooking skills.

HARDWARE REQUIREMENTS:

Processor: Intel i5 or above
Processor Speed: 2.0 GHz or higher
Random Access Memory: 8 GB or more
Hard Disk Memory: 500 GB
GPU: NVIDIA GTX 1650 or higher
Monitor: Colour Monitor with at least 1080p resolution

SOFTWARE REQUIREMENTS:

Operating System: Windows/Linux
Front End: React.js for a responsive user interface
Back End: Node.js and Express.js for server-side operations
Machine Learning Frameworks: TensorFlow and Keras for model development and training
Database: MongoDB or SQL for storing recipes and user data

CONCLUSION

The FIRE (Food Image to REcipe generation) model represents a significant advancement in the field of food computing, combining cutting-edge techniques from computer vision and natural language processing. By leveraging state-of-the-art models like Vision Transformers (ViT) for ingredient extraction, BLIP for title generation, and T5 for cooking instruction generation, FIRE successfully bridges the gap between visual understanding and natural language generation. It addresses the complex challenge of generating accurate, comprehensive recipes from food images by integrating multimodal techniques into a unified pipeline.

The model outperforms existing approaches in both ingredient prediction and recipe generation, demonstrating superior performance on large-scale datasets such as Recipe1M. With applications extending to personalized recipe customization and automated cooking systems, FIRE has the potential to revolutionize the way we interact with food-related technology.

REFERENCES

[1] Lukas Bossard, Matthieu Guillaumin, and Luc Van Gool. Food-101–mining discriminative components with random forests. In European conference on computer vision, pages 446–461. Springer, 2014.

[2] Micael Carvalho, R´emi Cad`ene, David Picard, Laure Soulier, Nicolas Thome, and Matthieu Cord. Cross-modal retrieval in the cooking context: Learning semantic text-image embeddings. In The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval, pages 35–44, 2018.

[3] Jing-jing Chen, Chong-Wah Ngo, and Tat-Seng Chua. Cross-modal recipe retrieval with rich food attributes. In Proceedings of the 25th ACM international conference on Multimedia, pages 1771–1779, 2017.

[4] Jing-Jing Chen, Chong-Wah Ngo, Fu-Li Feng, and Tat-Seng Chua. Deep understanding of cooking procedure for cross-modal recipe retrieval. In Proceedings of the 26th ACM international conference on Multimedia, pages 1020–1028, 2018.

[5] Mei Chen, Kapil Dhingra, Wen Wu, Lei Yang, Rahul Sukthankar, and Jie Yang. Pfid: Pittsburgh fast-food image dataset. In 2009 16th IEEE International Conference on Image Processing (ICIP), pages 289–292. IEEE, 2009.

[6] Gianluigi Ciocca, Paolo Napoletano, and Raimondo Schettini. Food recognition: a new dataset, experiments, and results. IEEE journal of biomedical and health informatics, 21(3):588–598, 2016.

[7] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In 2009 IEEE conference on computer vision and pattern recognition, pages 248–255. Ieee, 2009.

[8] Shaobo Fang, Chang Liu, Fengqing Zhu, Edward J Delp, and Carol J Boushey. Single-view food portion estimation based on geometric models. In 2015 IEEE International Symposium on Multimedia (ISM), pages 385–390. IEEE, 2015.

[9] Erica Greene. Extracting structured data from recipes using conditional random fields, Apr 2015.

[10] Hamid Hassannejad, Guido Matrella, Paolo Ciampolini, Ilaria De Munari, Monica Mordonini, and Stefano Cagnoni. Food image recognition using very deep convolutional networks. In Proceedings of the 2nd International Workshop on Multimedia Assisted Dietary Management, pages 41–49, 2016.

[11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 770–778, 2016.

[12] Ye He, Chang Xu, Nitin Khanna, Carol J Boushey, and Edward J Delp. Context based food image analysis. In 2013 IEEE International Conference on Image Processing, pages 2748–2752. IEEE, 2013.

[13] Luis Herranz, Weiqing Min, and Shuqiang Jiang. Food recognition and recipe analysis: integrating visual content, context and external knowledge. arXiv preprint arXiv:1801.07239, 2018.

[14] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. Densely connected convolutional networks. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 4700–4708, 2017.

THANK YOU

ANY

QUERIES?